BICOL UNIVERSITY COLLEGE OF SCIENCE

Computer Science and Information Technology

Department

IT 106 Application Development and Emerging Technologies

2nd Semester 2025-2026

Project Title:

Salespresso: A Web-based E-commerce System for Jazsam Coffee Store

Submitted to:

DR. JAYVEE CHRISTOPHER N. VIBAR

Faculty-in-charge

Submitted by:

MHEEA ROSE J. ANDES

FREDERICK B. BALDANO

EDGAR NEIL M. BAYOCA

JOSE MANUEL G. LABALAN

RYAN M. LUMBIS

BSIT 3C

TABLE OF CONTENTS

SECTION 1: INTRODUCTION

1.1 Background of the Project

Jazsam Coffee Store currently relies on manual pen-and-paper processes for handling orders and tracking sales, which often leads to long queues, overcrowded storefronts, and increased operational stress for staff. These traditional methods are prone to errors and inefficiencies, affecting both customer satisfaction and overall store performance.

Salespresso addresses these challenges by introducing a web-based e-commerce platform that supports a “pre-order and pick-up” system while simultaneously accommodating walk-in customers through a digitalized point-of-sale interface. This allows online customers to place orders in advance to avoid waiting, while enabling store staff to efficiently process on-site orders and receive real-time notifications for all incoming transactions. In addition, Salespresso provides tools for store staff to efficiently manage and monitor product lists, inventory levels, sales transactions, incoming orders, and customer rewards or discount programs.

By automating and integrating these key operations into one centralized platform, Salespresso can help improve order accuracy, reduce manual workload, and enhance overall store management. This digital solution enables Jazsam Coffee Store to streamline its daily operations, minimize delays and customer waiting times, improve customer satisfaction, and prevent errors and potential financial losses caused by manual processes.

1.2 Objectives

General Objective

Salespresso aims to develop a web-based e-commerce system that streamlines ordering, sales and inventory tracking, and store management processes for Jazsam Coffee Store by automating operations and implementing a hybrid system for pre-orders and walk-in transactions to improve efficiency, accuracy, and customer satisfaction.

Specific Objectives

1. To enable customers to place orders online in advance through a pre-order and pick-up system to reduce waiting time and long queues.
2. To provide a streamlined interface for staff to record and manage walk-in orders, ensuring all sales data is synchronized in real-time in a single system.
3. To provide real-time order notifications for store staff to ensure timely preparation of customer orders and improve order accuracy.
4. To automate the management of product listings and inventory levels to ensure accurate tracking of available items.
5. To automate the recording and monitoring of sales transactions to minimize errors and improve financial tracking.
6. To integrate a customer rewards and discount system to enhance customer engagement and retention.

1.3 Scope and Limitations

Scope

1. Provides an online platform where customers can browse products, view details, prices, and stock availability.
2. Allows customers to place orders through a pre-order and pick-up system with a guided checkout process.
3. Enables store staff to input and process orders for walk-in customers directly into the system to centralize store transactions.
4. Records online payments to ensure accurate tracking of completed transactions.
5. Enables order status monitoring (e.g., pending, preparing, completed) with real-time order notifications for store staff.
6. Automates product listing and inventory management, including adding, updating, and removing items for accurate monitoring of stock levels.
7. Records and manages sales transactions to improve accuracy and financial tracking.
8. Integrates a customer rewards and discount system to enhance engagement and retention.

Limitations

1. Designed for a single coffee store only and does not support multi-vendor or multi-branch operations.
2. Does not include AI-based product recommendations or personalized suggestions.
3. Does not support real-time GPS tracking for customers or deliveries; only order status updates are provided.
4. Focused only on pre-order and pick-up; delivery management and logistics are not included.
5. Dependent on stable internet connectivity for both customers and administrators to access system features.

SECTION 2: METHODS

2.1 Development Methodology

The project follows the Agile Software Development Life Cycle (SDLC), which emphasizes flexibility, iterative development, and continuous user feedback throughout the development process. Agile breaks down the development process into smaller cycles called sprints, allowing the system to be developed incrementally rather than in a single release.

In this approach, system requirements are organized into a product backlog, where key features such as online ordering, inventory management, and sales tracking are prioritized. During each sprint, selected features are designed, developed, and tested within a short time frame. This allows continuous improvement and early detection of issues.

Activities such as sprint planning, and sprint reviews, and daily stand-ups are conducted to ensure alignment with project goals and to adapt to any changes in requirements. Feedback from users is incorporated after each iteration, ensuring that the system remains relevant, user-friendly, and aligned with the needs of Jazsam Coffee Store.

2.2 Tools and Technologies Used

Front-end

- React – Used for building the user interface and interactive components.
- CSS – Used for styling and designing the layout of the application.
- Javascript – Handles client-side logic and interactivity.

Back-end

- PHP – Handles server-side logic, processing requests, and communication between the front-end and database.

Database

- MySQL – Stores and manages system data such as products, orders, user accounts, and transactions.

Platform

- Web Browser – Serves as the primary platform for accessing the system, allowing users to interact with the system through internet browsers (e.g., Google Chrome, Microsoft Edge) without requiring installation.

2.3 System Design Overview

Salespresso is designed as a web-based e-commerce system that supports both customers and store administrators through an integrated platform. The system follows a structured yet flexible design aligned with Agile principles, ensuring that each component can be improved incrementally.

The system consists of two main user roles: customers and administrators (store staff). Customers can browse products, place pre-orders, make payments, and track order status through a user-friendly interface. Furthermore, the system is also designed to support walk-in customers, where store staff can manually input orders into the system as they are received at the counter, ensuring that the inventory and sales data remain synchronized across all sales channels. On the other hand, administrators manage product listings, monitor inventory levels, process incoming orders, and track sales transactions through an administrative dashboard.

The architecture of the system is composed of key functional modules, including product management, order processing, payment recording, inventory tracking, and sales reporting. These modules interact with a centralized database to ensure that both customers and staff have access to accurate and up-to-date information.

Additionally, the system incorporates a notification mechanism that alerts staff of new orders, enabling timely preparation and reducing delays. The integration of a rewards and discount feature further enhances customer engagement.

Overall, the system design focuses on improving operational efficiency, ensuring data accuracy, and providing a seamless experience for both customers and store administrators.

SECTION 3: HYPOTHETICAL LICENSING MODEL

Proposed Licensing Model: Subscription Software-as-a-Service (SaaS)

If Salespresso were to be publicly released and deployed within a cloud hosting environment, adopting a Subscription Software-as-a-Service (SaaS) model would be the most practical and scalable approach. Under this framework, Jazsam Coffee Store would pay a recurring monthly or annual subscription fee to maintain access to the centralized web-based e-commerce platform.

Justification for the SaaS Model:

- **Sustainable Maintenance and Continuous Value:** The recurring revenue generated by the subscription ensures the sustainability of continuous cloud hosting and database management. Furthermore, it financially supports ongoing system maintenance, bug fixes, and incremental feature enhancements deployed through the project's Agile Scrum framework.
- **Cost-Effective and Targeted Scope:** Because the architecture of Salespresso is strictly tailored for single-store operations—explicitly excluding multi-vendor or multi-branch capabilities—the operational overhead remains relatively low. Consequently, the SaaS subscription fee can be kept highly affordable and proportionate to the budget constraints of a small-to-medium enterprise.
- **Frictionless Accessibility and Reliability:** A cloud-hosted SaaS model is optimal for supporting the system's hybrid service approach. It guarantees stable, high-speed, and continuous availability of the online pre-order interface, ensuring that both walk-in customers and one-time online users experience a seamless transaction process without the friction of mandatory account creation.

SECTION 4: DEPLOYMENT

4.1 Deployment Environment

The system's deployment environment consists of a two-phase approach: a local server environment for prototyping and a cloud hosting environment for production.

- **Local Server (Prototype):** The prototyping phase utilizes a local machine setup via localhost. This environment relies on Node.js to run the React development server and XAMPP to provide a local Apache server and MySQL database.
- **Cloud Hosting (Production):** The future production environment will be hosted on Vercel. Vercel is a cloud platform specifically optimized for modern front-end frameworks.

4.2 Deployment Steps

The system deployment is carried out in two distinct phases:

Phase 1: Local Deployment (Prototyping via Localhost)

- **Step 1: Install Required Tools.** Set up the development environment by installing Node.js, XAMPP, and Visual Studio Code.
- **Step 2: Clone the Repository.** Use the command *git clone <repository-url>* to clone the source code from GitHub to the local machine.
- **Step 3: Install Front-End Dependencies.** Navigate to the project directory and run *npm install* to install all necessary React dependencies.
- **Step 4: Configure the Database.** Use phpMyAdmin via XAMPP to set up the MySQL database and import the provided SQL file to create tables and initial data.
- **Step 5: Configure Environment Variables.** Create a .env file in the project's root directory to store configurations like the database connection and the local API base URL pointing to the local XAMPP server (e.g., *http://localhost/api*).
- **Step 6: Run the Development Server.** Start the React application locally by running npm start, which makes the system accessible at *http://localhost:5173* for testing.

Phase 2: Production Deployment (Future – Vercel)

- **Step 1: Prepare the Production Build.** Generate an optimized build of the application using the *npm run build* command prior to deployment.
- **Step 2: Connect Repository to Vercel.** Link the GitHub repository to a Vercel account. Vercel will then automatically detect the React framework and set up the build configurations accordingly.
- **Step 3: Configure Environment Variables on Vercel.** Add the production environment variables, such as the live API base URL, via the Vercel dashboard.
- **Step 4: Deploy the Application.** Pushing code to the main branch of the repository will automatically trigger a deployment on Vercel. Once complete, the system will be available to end users at a live URL (e.g., *https://project-name.vercel.app*).
- **Step 5: Monitor and Maintain.** Utilize Vercel's built-in analytics and deployment logs to track uptime, monitor performance, and check for errors post-deployment.

4.3 Access Information

GitHub Repository Link: https://github.com/ItsRyan504/ADET-Final-Project

SECTION 5: SOURCE CODE

5.1 Repository Link

GitHub Repository Link: https://github.com/ItsRyan504/ADET-Final-Project

5.2 Code Structure

```
├── src/
│   ├── JazSam/                ← React frontend
│   │   ├── src/
│   │   │   ├── pages/         ← 11 pages (Home, Menu, Login, etc.)
│   │   │   ├── components/    ← Navbar, Footer, Chatbot, etc.
│   │   │   ├── context/       ← AuthContext, OrdersContext, StoreContext
│   │   │   └── App.jsx        ← Router & entry point
│   │   ├── tests/             ← Python pytest suite (6 files)
│   │   └── salespresso_db.sql ← Database schema
│   │
│   └── salespresso-api/       ← PHP backend
│       ├── db.php             ← Database connection
│       ├── auth.php           ← Admin authentication
│       ├── customers.php      ← Customer registration/login
│       ├── orders.php         ← Order CRUD
│       ├── orders-stream.php  ← Real-time SSE updates
│       ├── products.php       ← Product management
│       ├── ingredients.php    ← Inventory
│       ├── rewards.php        ← Loyalty system
│       ├── payment.php        ← Payment processing
│       └── paymongo-*.php     ← PayMongo integration
```

5.3 Key Functional Components

1. Authentication

- Customers: Email/password with rate limiting and brute-force protection
- Admins: JWT token-based auth with role access (ProtectedAdminRoute)
- Sessions stored in localStorage

2. Order Management

- Pickup and delivery orders with notes and address

- Real-time status updates via Server-Sent Events (SSE)
- Admin dashboard to move orders: Pending → Preparing → Completed
- Audio notification when an order is ready

3. Loyalty & Rewards System

- Points earned per order (1point = 1 order)
- Birthday rewards with special claim mechanics
- Points redeemable on checkout

4. Product & Inventory

- Full product CRUD with size variants, pricing, and temperature options
- Category filtering (Coffee, Milktea, Matcha, Soda, etc.)
- Ingredient-based recipe templates with low-stock alerts
- Image upload with crop modal

5. Payment Integration

- PayMongo checkout API (card payments)
- Cash on delivery support
- Webhook handling for payment status updates

6. Admin Dashboard

- Real-time order queue with audio alerts
- Employee management with roles
- Audit logging for all admin actions
- Birthday reward management

SECTION 6: DEVELOPMENT NARRATIVE / PROJECT JOURNAL

This section documents the development process of Salespresso, detailing the roles, responsibilities, and collaborative efforts of the team throughout the project lifecycle.

6.1 Team Composition and Roles

Mheea Rose J. Andes — *Client/Frontend Developer, Database Administrator, Tester*

Designed and developed the web-based client HTML interface. Handled the creation and management of the MySQL database tables, fields, and relationships. Conducted API response testing and ensured clear, readable JSON data display on the frontend.

Frederick B. Baldano — *Integration Specialist, QA Tester*

Handled the database connection setup in db.php using PDO to ensure secure communication. Managed authentication handling and Bearer token enforcement on protected routes. Validated API endpoints for customer registration, login, and error prevention.

Edgar Neil M. Bayoca — *Code Reviewer, Backend Assistant, QA Tester*

Monitored codebase integrity and GitHub repository structure. Specialized in backend security, implementing strict server-side input validations (e.g., isValidGrade) to prevent data corruption, SQL injections, and system crashes from edge cases.

Jose Manuel G. Labalan — *Project Manager, Backend Developer, Tester*

Acted as the team leader, managed role delegation, and maintained a high-level overview of project deliverables. Conceptualized the initial relational database structure and developed the foundational HTTP method routing logic within api.php. Ensured synchronization between frontend and backend teams.

Ryan M. Lumbis — *Backend Developer, Client Developer, Tester*

Implemented PHP to MySQL database connections with integrated JSON error handling. Assisted in developing the backend CRUD logic for student and product

management. Improved the terminal flow and user interactions for the client-side applications.

6.2 Project Journal

Date: May 20, 2026

Activities Done: Initiated the Automated Software Testing phase using Python 3.13, pytest, and the requests library. The team divided the testing workload to validate all core API endpoints of the Salespresso system, including Products CRUD, Customer Authentication, Input Validation, Rewards Management, and Admin Sessions. Executed comprehensive test suites for both public and protected routes.

Progress Achieved: Successfully completed functional, security, and authorization testing across the system. Validated that the backend properly handles correct HTTP methods, enforces Bearer token authentication for admin actions, and securely interacts with the MySQL database without crashing under stress.

Problems Encountered:

1. **Rate Limiting Roadblocks:** Automated test scripts executed too quickly, rapidly exhausting the API's rate limits (e.g., 5 admin login attempts per 5 minutes, 5 customer registrations per hour). This triggered false-positive HTTP 429 Too Many Requests errors across the suite.
2. **Session Overwrites:** Discovered that the issueAdminToken() function only supports one active token per staff member. Logging in from an automated test displaced existing admin sessions.
3. **Validation & Error Code Gaps:** Found that customer registration lacked strict email format validation, allowing malformed strings to bypass filters. Additionally, invalid inputs (like oversized strings or incorrect data types) were handled gracefully without crashing, but returned permissive HTTP 200 OK statuses instead of standard REST API client error codes (e.g., 400 or 422). 4. **Test Suite**

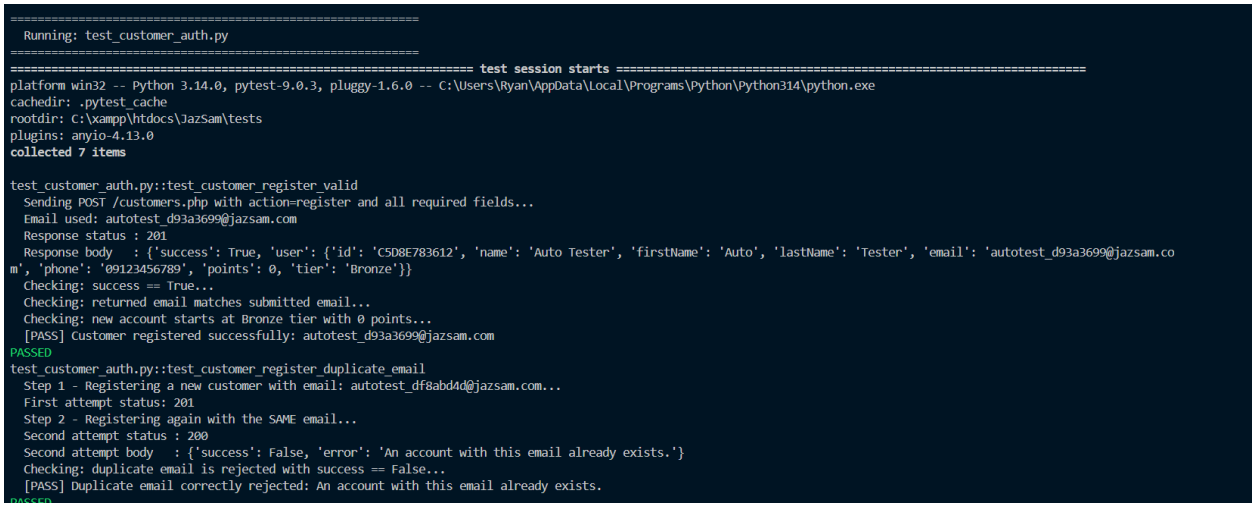
Warnings: Encountered PytestReturnNotNoneWarning alerts due to unnecessary return statements in early script drafts.

Solutions Applied:

1. **Rate Limit Exemption & Fixtures:** Coordinated to implement a localhost IP exemption within rate_limit.php to prevent the testing environment from being blocked. Utilized session-scoped fixtures (admin_token) in conftest.py with autouse=True to share a single login token across all tests, drastically reducing redundant API calls.
2. **Stable Test Accounts:** Utilized a pre-seeded stable test account (autotest_stable@jazsam.com) to bypass registration limits while testing authentication flows. Documented the single-session limitation for potential future upgrades.
3. **Strict Server-Side Filters:** Recommended integrating filter_var($email, FILTER_VALIDATE_EMAIL) for robust server-side data sanitization, and flagged the HTTP 200 error-handling behavior for future refactoring toward stricter HTTP status code usage. 4. **Script Optimization:** Cleaned up Python test scripts by removing unnecessary return statements to resolve Pytest warnings.

Screenshots of Automated Software Testing:

Customer testing:



```
PASSED
test_customer_auth.py::test_customer_login_valid
  Sending POST /customers.php with action=login using stable test account...
  Email used: autotest_stable@jazsam.com
  Response status : 200
  Response body   : {'success': True, 'user': {'id': 'C2368869F06', 'name': 'Auto Tester', 'firstName': 'Auto', 'lastName': 'Tester', 'email': 'autotest_stable@jazsam.com'
, 'phone': '09123456789', 'points': 0, 'tier': 'Bronze', 'profilePicture': None}}
  Checking: success == True...
  Checking: returned email matches login email...
  [PASS] Customer login successful: autotest_stable@jazsam.com
PASSED
test_customer_auth.py::test_customer_login_wrong_password
  Sending POST /customers.php with correct email but WRONG password...
  Response status : 200
  Response body   : {'success': False, 'error': 'Incorrect password. Please try again.'}
  Checking: success == False...
  [PASS] Wrong password correctly rejected: Incorrect password. Please try again.
PASSED
test_customer_auth.py::test_customer_login_nonexistent_email
  Sending POST /customers.php login with an email that was never registered...
  Response status : 200
  Response body   : {'success': False, 'error': 'No account found with this email.'}
  Checking: success == False...
  [PASS] Non-existent email correctly rejected: No account found with this email.
PASSED
```

Products testing:

```
============================================================
  Running: test_products.py
============================================================
============================= test session starts ==============================
platform win32 -- Python 3.14.0, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\Ryan\AppData\Local\Programs\Python\Python314\python.exe
cachedir: .pytest_cache
rootdir: C:\xampp\htdocs\JazSam\tests
plugins: anyio-4.13.0
collected 7 items

test_products.py::test_get_products_returns_list
  Sending GET /products.php with no authentication (public endpoint)...
  Response status : 200
  Checking: status is 200...
  Checking: response is a list (not a dict or error)...
  [PASS] GET products returned 43 product(s)
PASSED
test_products.py::test_get_products_structure
  Fetching products and checking the structure of each item...
  Required fields to check: {'name', 'status', 'price', 'id', 'category'}
  Checking product 'Americano' for missing fields...
  Checking product 'AutoTest Brew' for missing fields...
  Checking product 'AutoTest Brew' for missing fields...
  [PASS] Product structure validated for 3 product(s)
PASSED
test_products.py::test_post_product_requires_auth
  Sending POST /products.php with NO Authorization header...
  Response status : 401
  Checking: server returns 401 (Unauthorized)...
  [PASS] Unauthenticated product creation correctly rejected (401)
PASSED
```

```
test_products.py::test_post_product_missing_name
  Sending POST /products.php with admin token but NO 'name' field...
  Response status : 400
  Response body   : {'error': 'Product name is required.'}
  Checking: server rejects request and returns an error...
  [PASS] Missing product name correctly rejected
PASSED
test_products.py::test_post_product_valid
  Sending POST /products.php with admin token and all valid fields...
  Product ID used : PE7DBBFB0
  Response status : 201
  Response body   : {'id': 'PE7DBBFB0'}
  Checking: status is 200 or 201 and response contains 'id'...
  [PASS] Product created successfully: PE7DBBFB0
PASSED
```

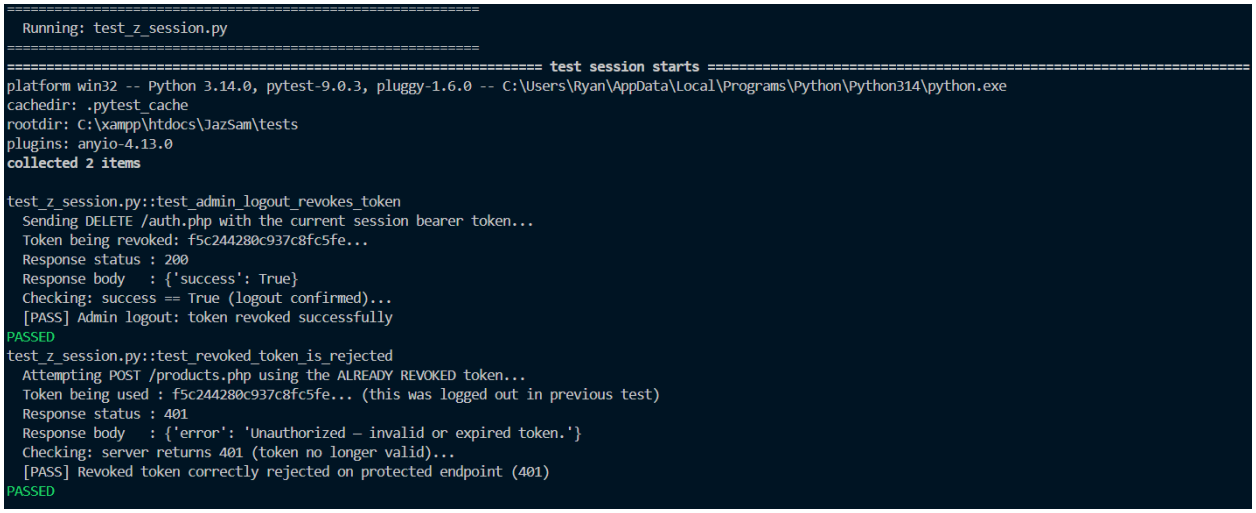
Rewards testing:

```
============================================================
  Running: test_rewards.py
============================================================
============================= test session starts ==============================
platform win32 -- Python 3.14.0, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\Ryan\AppData\Local\Programs\Python\Python314\python.exe
cachedir: .pytest_cache
rootdir: C:\xampp\htdocs\JazSam\tests
plugins: anyio-4.13.0
collected 8 items

test_rewards.py::test_get_rewards_returns_list
  Sending GET /rewards.php with no authentication (public endpoint)...
  Response status : 200
  Checking: status is 200 and response is a list...
  [PASS] GET rewards returned 17 reward(s)
PASSED
test_rewards.py::test_get_rewards_structure
  Fetching rewards and validating each item has the required fields...
  Required fields to check: {'stamps', 'id', 'name', 'type', 'value'}
  Checking reward 'Discount' for missing fields...
  Checking reward 'AutoTest 10% Off' for missing fields...
  Checking reward 'AutoTest 10% Off' for missing fields...
  [PASS] Reward structure validated for 3 reward(s)
PASSED
test_rewards.py::test_post_reward_requires_auth
  Sending POST /rewards.php with NO Authorization header...
  Response status : 401
  Checking: server returns 401 (Unauthorized)...
  [PASS] Unauthenticated reward creation correctly rejected (401)
PASSED
```

Input validating testing:

```
============================================================
  Running: test_input_validation.py
============================================================
============================= test session starts ==============================
platform win32 -- Python 3.14.0, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\Ryan\AppData\Local\Programs\Python\Python314\python.exe
cachedir: .pytest_cache
rootdir: C:\xampp\htdocs\JazSam\tests
plugins: anyio-4.13.0
collected 10 items

test_input_validation.py::test_invalid_method_on_products
  Sending PATCH /products.php (PATCH is not a supported method)...
  Response status : 405
  Checking: server returns 405 Method Not Allowed...
  [PASS] PATCH /products.php -> 405 Method Not Allowed
PASSED
test_input_validation.py::test_invalid_method_on_rewards
  Sending PATCH /rewards.php (PATCH is not a supported method)...
  Response status : 405
  Checking: server returns 405 Method Not Allowed...
  [PASS] PATCH /rewards.php -> 405 Method Not Allowed
PASSED
test_input_validation.py::test_login_with_sql_injection_email
  Sending POST /auth.php with SQL injection string as email...
  Injection input : ' OR '1'='1
  Response status : 200
  Response body   : {'success': False, 'error': 'Invalid credentials. Please try again.'}
  Checking: login is rejected and server did NOT crash (no 500)...
  [PASS] SQL injection safely rejected (HTTP 200)
PASSED
```

Z session testing:



6.3 Individual Quality Assurance Reports

Mheea Rose J. Andes — Products API Testing

- **Focus:** Validated the public accessibility of the product catalog and enforced strict authorization for write operations.
- **Key Tests Executed:** Verified successful retrieval of all 43 products (HTTP 200), successfully blocked unauthenticated product creation attempts (HTTP 401), and confirmed admins could seamlessly create new products and receive database IDs (HTTP 201).
- **Findings:** The Products API is secure and functionally stable. Centralizing the admin login into a session-scoped fixture made the test suite significantly faster and prevented rate-limit exhaustion.

Frederick B. Baldano — Customer Registration & Login

- **Focus:** Validated account creation, duplicate email prevention, and missing field handling.

- **Key Tests Executed:** Verified new customers receive a Bronze tier account with 0 points upon successful registration. Confirmed the system blocks duplicate emails and rejects registrations missing required fields like names or passwords.
- **Findings:** Automated testing effectively surfaced hidden bugs missed during manual testing, specifically the lack of strict email format validation before database insertion.

Edgar Neil M. Bayoca — Input Validation & Error Handling

- **Focus:** Stressed the system's defenses using edge cases, oversized inputs, and security vulnerabilities.
- **Key Tests Executed:** Executed SQL injection strings (' OR '1'='1) into email fields, submitted 500-character inputs, and tested incorrect data types (integers instead of strings).
- **Findings:** The backend is highly robust; it successfully rejected all malicious and malformed inputs without throwing catastrophic HTTP 500 server crashes. Identified a need to transition the API's failure responses from HTTP 200 to standard client error codes.

Jose Manuel G. Labalan — Rewards API Testing

- **Focus:** Automated the complete CRUD lifecycle for the store's discount and free-item systems.
- **Key Tests Executed:** Sequentially created, updated, and deleted rewards to ensure accurate database state changes. Validated required response structures (id, name, type, value, stamps) and tested HTTP 404 handling for non-existent items.
- **Findings:** The Rewards system safely handles all operations. The testing process underscored the importance of managing shared state to avoid triggering server defense mechanisms during automated QA.

Ryan M. Lumbis — Admin Authentication

- **Focus:** Validated the foundational security of the admin panel through token issuance and credential verification.
- **Key Tests Executed:** Acquired valid Bearer tokens for the test session. Verified the system actively rejects wrong passwords, non-existent staff emails, and empty JSON request bodies with descriptive error messages.
- **Findings:** Flagged a behavioral limitation where switching devices or initiating a new session wipes out the previous admin token. This highlighted how authentication design decisions directly impact real-world staff usability.